



FIREBRAND

LYCEUM EDUCATION GROUP

User Stories and Design Documentation

Turning a system description into a plan you can actually build from.

What You Will Be Able To Do By This Afternoon

Six concrete planning skills you will practise and apply today.

1

Translate a system description: pull out the separate needs hiding inside one paragraph of requirements.

2

Write proper user stories: use the As a, I want, so that structure with real sector examples.

3

Write acceptance criteria: define exactly what "done" looks like for a story.

4

Sketch pseudocode: describe a piece of logic in plain steps before writing real JavaScript.

5

Produce a simple design note: capture inputs, outputs and rules so another developer could pick up your plan.

6

Explain your plan to two audiences: a technical teammate and a non-technical stakeholder.

The Vague Brief

A stakeholder hands your team a single paragraph and walks away.

1

CONTEXT

A retail company wants a new checkout page. A product manager sends one message: "We need something that stops people checking out if their basket is empty and shows a discount for loyalty members."

2

CHALLENGE

Three different developers read that sentence and each build something slightly different.

3

TASK

Read the brief silently and jot down, in your own words, what you think it is actually asking for.

One Sentence, Many Decisions

A single request can contain several separate rules that need separate answers.

- 1 What counts as empty: zero items, or zero items and zero pounds?
- 2 When does the check happen: before checkout starts, or when the button is pressed?
- 3 What message does the shopper see: nothing, an error, or a redirect?
- 4 Who counts as a loyalty member: an account flag, a stored number, something else?
- 5 Is the discount automatic or does it need a code: two very different builds.

Where This Sits in the Life Cycle

User stories and design documents belong to one specific stage of the life cycle you already know.

1

REQUIREMENTS

understanding what is needed and who it is for, in plain language.

2

DESIGN

turning the requirement into a plan, a user story, acceptance criteria and pseudocode.

3

BUILD

writing the actual JavaScript that satisfies the design.

4

TEST

checking the build against the acceptance criteria written at the design stage.

5

HANDOVER

passing on the finished work with the documentation that explains it.

Who Writes What

Product owner or business analyst

- Gathers the original need from the business or the client
- Writes the first draft of a user story
- Prioritises which stories get built first

Developer

- Writes or refines acceptance criteria with technical detail
- Sketches pseudocode and a short design note
- Builds the code that satisfies the criteria

Tester

- Reads the acceptance criteria written at design stage
- Checks the finished feature against each criterion
- Reports back any criterion that is not met

The User Story Shape

The pattern

As a [type of user], I want [a goal], so that [a reason].

Three Worked User Stories

Healthcare

As a receptionist, I want the system to flag a missed appointment, so that I can follow up with the patient.

Financial services

As an account holder, I want to see a warning before a payment takes my balance negative, so that I avoid an unexpected overdraft.

Retail

As a loyalty member, I want to see my discount applied automatically at checkout, so that I do not need to remember a code.

Logistics

As a warehouse operator, I want low stock items flagged on the report, so that I can reorder before running out.

Spot the Missing Piece

Two attempts at the same user story, one missing a part of the pattern

draft-story-v1.txt

```
1 As a patient, I want to book an
2 appointment online.
3 so that I do not need to phone
4 the surgery.
```

draft-story-v2.txt

```
1 As a patient, I want to book an
2 appointment online,
3 so that I do not need to phone
4 the surgery.
```

Complete the Story

Add the missing "so that" clause to three incomplete user stories.

- 1 Open the incomplete-stories.txt starter file provided.
- 2 Write a plausible "so that" clause for each of the three stories.
- 3 Compare your reasons with a partner's for the same story.



TIMEBOX

8 minutes



OUTPUT

**Three completed user stories,
each with all three parts
present.**

A Story Is Not Enough

“

A user story tells you what someone wants. It does not tell you when you are finished.

The gap every team hits after writing their first story.

Acceptance Criteria: Defining Done

Acceptance criteria are the specific, checkable conditions that prove a story is finished.

- 1 Written as short, testable statements: each one is either true or false when checked.
- 2 Cover the normal case: what should happen when everything is as expected.
- 3 Cover edge cases: what should happen at a boundary, like exactly zero or exactly the limit.
- 4 Cover the guard case: what should happen when the input is invalid or missing.
- 5 Owned jointly: written by a developer, checked later by a tester against the same list.

One Story, Four Criteria

Acceptance criteria written underneath the loyalty discount story from earlier.



loyalty-story.txt

```
1 As a loyalty member, I want to see my discount
2 applied automatically at checkout, so that I do
3 not need to remember a code.
4 Given a loyalty member, discount shows at checkout
5 Given a non-member, no discount is shown
6 Given a loyalty member with empty basket, no discount shown
7 Discount amount matches the stored member rate
```

Which Criteria Are Missing

Read a user story and its acceptance criteria, then spot a gap.

- 1 Open story-gap-check.txt showing a delivery flag story with three criteria.
- 2 Identify one realistic situation the three criteria do not cover.
- 3 Write a fourth acceptance criterion covering that situation.



TIMEBOX

8 minutes



OUTPUT

One additional, testable acceptance criterion in plain English.

The Same Gap, Every Sector

Easy to miss

- A payment exactly at the account limit
- A patient exactly at the appointment cutoff time
- A stock count of exactly zero, not below zero

Why it matters

- Boundary values are where off-by-one mistakes hide
- A vague criterion lets a boundary slip through untested
- Naming the boundary in writing forces a decision before code exists

What Pseudocode Actually Is

Pseudocode describes the steps of a solution in plain language before real code exists.

- 1 Not a programming language: no exact JavaScript syntax rules apply.
- 2 Still precise: each line should describe one clear, unambiguous step.
- 3 Written to be read by anyone: a non-technical reviewer should follow it too.
- 4 A bridge, not a detour: it should map closely onto code you could genuinely write.

From Plain English to Pseudocode

pseudocode

```
1 CHECK appointment status
2 IF appointment date is today
3   SHOW "Reminder due today"
4 ELSE IF appointment date has passed
5   SHOW "Missed, follow up needed"
6 ELSE
7   SHOW "No action needed yet"
```

1

Check appointment status

decide whether a reminder should be sent

2

Compare to today

read the appointment date against today's date

3

Branch on the result

choose the message shown to the receptionist

Predict the JavaScript

Match each pseudocode line to the real JavaScript keyword it becomes.

- 1 Open pseudocode-match.txt showing five plain-language lines.
- 2 Write the JavaScript keyword each line maps onto.
- 3 Check your answers against a partner's.



TIMEBOX

7 minutes



OUTPUT

Five pseudocode lines each paired with a JavaScript keyword.

Try Writing Pseudocode Yourself

The story and criteria you will turn into pseudocode next.



stock-story.txt

```
1 As a warehouse operator, I want stock below ten
2 units flagged as low, so that I can reorder before
3 running out.
4 Given stock count under 10, item flagged as low
5 Given stock count of exactly 10, item is not flagged
6 Given stock count above 10, item is not flagged
```

Write the Pseudocode

Turn the stock story and its criteria into three lines of pseudocode.

- 1 Use the story and criteria shown on the previous slide.
- 2 Write an IF and ELSE structure describing the flagging logic.
- 3 Name which JavaScript comparison operator each branch needs.



TIMEBOX

10 minutes



OUTPUT

A short pseudocode block with the correct boundary comparison.

Same Pattern, Three Sectors

Retail stock flag

- IF stock count is less than 10
- SHOW "Low stock, reorder needed"
- ELSE SHOW "Stock level fine"

Healthcare triage flag

- IF wait time is greater than 240 minutes
- SHOW "Priority review needed"
- ELSE SHOW "Within target wait"

What a Design Note Adds

A design note gathers the story, criteria and pseudocode into one short document.

1

Inputs: what data or values the feature reads or receives.

2

Outputs: what the feature shows, stores or sends as a result.

3

Rules: the plain-English logic, usually the pseudocode already written.

4

Notes: anything a future developer should know, including edge cases considered.

A Complete Design Note

The full design note for the stock flag feature, ready to build from.



stock-flag-design.txt

```
1 Inputs: stockCount (number), itemName (string)
2 Outputs: a status message shown on the page
3 Rules: if stockCount is less than 10, flag as low
4 Rules: if stockCount is 0, flag as out of stock
5 Notes: exactly 10 units does not count as low
```

The Morning Build: Plan a Feature End to End

Take one system description and produce a full set of planning artefacts.

- 1 Open morning-build-brief.txt describing a delivery status feature.
- 2 Write one complete user story using the As a, I want, so that shape.
- 3 Write three acceptance criteria including one boundary case.
- 4 Write three lines of pseudocode and a four-part design note.



TIMEBOX

18 minutes



OUTPUT

One user story, three acceptance criteria, pseudocode and a design note.

Check Your Own Work



My story names who the feature is for, what they want, and why

Uses the As a, I want, so that shape completely



My criteria are testable

Each one can be checked with a clear yes or no



Have I named the boundary value explicitly

Check the exact 2 hour case is covered, not just above and below it



Does my pseudocode map onto real JavaScript

Check every line could become an if, else if or else you already know

Naming Something You Missed

- Which part of the morning build did you find hardest to write clearly?
- Was it the reason in the user story, a testable criterion, or the pseudocode itself?
- What would help you get better at that specific part next time?

Discuss briefly with a partner before the morning recap.

Morning in One Line



A user story says what and why, acceptance criteria say when it is done.



FIREBRAND

LYCEUM EDUCATION GROUP

Lunch

Back at the top of the hour, bring your morning build notes with you.



FIREBRAND

LYCEUM EDUCATION GROUP

Welcome Back

This afternoon: a richer scenario, sharper criteria, and a full planning pack.

A Longer Brief

A financial services client sends a longer description than this morning's brief.

1

CONTEXT

A challenger bank wants customers to see a spending alert. "Customers should be warned if a single payment is unusually large for their normal spending, and also warned separately if it would take their balance below zero. Staff should be able to see which customers have had either warning this month."

2

CHALLENGE

This single paragraph actually describes at least three separate features.

3

TASK

Read the brief and list, in your own words, how many separate features you can find.

One Brief, Several Stories

A single system description usually splits into more than one user story.

1

READ THE BRIEF

find every distinct thing a different type of user wants.

2

SEPARATE THE USERS

a customer and a staff member likely need different stories.

3

WRITE ONE STORY PER NEED

resist cramming two goals into one story.

4

ORDER THEM

which story matters most if only one gets built first.

Two Stories from One Brief

Two of the three stories hiding inside the challenger bank brief.



Large payment warning

As a customer, I want a warning shown for an unusually large payment, so that I can catch a mistake or fraud early.



Low balance warning

As a customer, I want a separate warning when a payment would take my balance below zero, so that I avoid an unexpected overdraft.

Write the Missing Story

Write the third user story hiding inside the challenger bank brief.

- 1 Reread the brief on the earlier scenario slide.
- 2 Write a user story for the staff member, not the customer.
- 3 Check your story names who, what and why clearly.



TIMEBOX

7 minutes



OUTPUT

One complete user story written from the staff member's point of view.

Naming the Ambiguity

Worth raising with the stakeholder

What exactly counts as an unusually large payment is not defined anywhere in the brief.

Acceptance Criteria for Both Warnings

Acceptance criteria for the large payment story, ready to critique next.



large-payment-criteria.txt

- 1 Given a payment over 3x average spend, warning shown
- 2 Given a payment at or under average spend, no warning
- 3 Given a new customer with no spending history, no warning
- 4 Given the warning threshold, staff can see it was shown

Judge These Criteria

Decide whether a second set of acceptance criteria is genuinely testable.

- 1 Open balance-criteria-review.txt showing four proposed criteria.
- 2 Mark each one as testable or too vague to check.
- 3 Rewrite any vague criterion so it becomes testable.



TIMEBOX

10 minutes



OUTPUT

**Four criteria each marked,
with any vague ones
rewritten.**

Vague Versus Testable

Too vague to check

- The warning should be clear and helpful
- Staff should understand what happened easily

Genuinely testable

- Given balance below zero after payment, warning text reads "This payment will take your balance below zero"
- Given balance at exactly zero, no warning is shown

Pseudocode for Both Warnings

pseudocode

```
1 CHECK payment vs threshold
2 IF over threshold
3   SHOW "Large payment
4   warning"
5 CHECK balance after pay
6 IF balance < 0
7   SHOW "Low balance
8   warning"
```

1

Threshold check

compare payment size to the spend limit

2

Balance check

compare balance after payment to zero

3

Two warnings

both can fire on the same payment

Explaining a Plan, Two Ways

“

The same plan needs two different sentences depending on who is listening.

A skill this cohort has practised since Day 3.

The Same Story, Two Audiences

Explain Your Morning Build Twice

Write two short explanations of your morning delivery flag build.

- 1 Reread your morning build's story, criteria and pseudocode.
- 2 Write a two sentence technical explanation for a developer.
- 3 Write a two sentence plain explanation for a non-technical manager.



TIMEBOX

10 minutes



OUTPUT

Two short written explanations of the same feature, for two audiences.

What Counts as Jargon Here

A quick reminder of which words belong in the technical version only.

1

Comparison operator, threshold, boundary case: developer vocabulary from today.

2

Acceptance criteria, pseudocode, user story: names for planning artefacts themselves.

3

If, else if, else, flag, condition: JavaScript concepts the manager does not need.

From Plan to Project Studio

Today's four artefacts are exactly what Day 15's project studio expects you to bring.

User story

the feature you are choosing to build, and why it matters.

Acceptance criteria

how you and a peer will judge whether it works.

Pseudocode

the branching logic sketched out before you open your editor.

Design note

inputs, outputs and rules, so a partner could follow your plan.

Choosing Your Own Feature

Four starting briefs to choose from for the afternoon's main build.



Healthcare

A GP surgery wants overdue vaccination reminders flagged on a patient's record.



Financial services

A bank wants a warning shown before a transfer would breach a daily limit.



Retail

An online store wants an urgent restock flag when an item's stock reaches zero.



Security

A building access system wants a flag raised after three failed entry attempts.

The Afternoon Build: A Complete Planning Pack

Produce a full planning pack for one feature, ready for the project studio.

- 1 Choose one brief from the four options just shown.
- 2 Write one user story and four acceptance criteria including a boundary.
- 3 Write four lines of pseudocode covering every branch.
- 4 Write a four part design note: inputs, outputs, rules, notes.



TIMEBOX

20 minutes



OUTPUT

A complete planning pack: story, criteria, pseudocode and design note.

Peer Review Against the Pack

Swap planning packs with a partner and check them against four criteria.

- 1 Swap your completed planning pack with a partner.
- 2 Check their story has all three parts and their criteria are testable.
- 3 Check their pseudocode covers every branch their criteria describe.
- 4 Give one specific piece of praise and one specific suggestion.



TIMEBOX

10 minutes



OUTPUT

Written peer feedback on a partner's planning pack, with one suggestion.

What Good Feedback Sounds Like

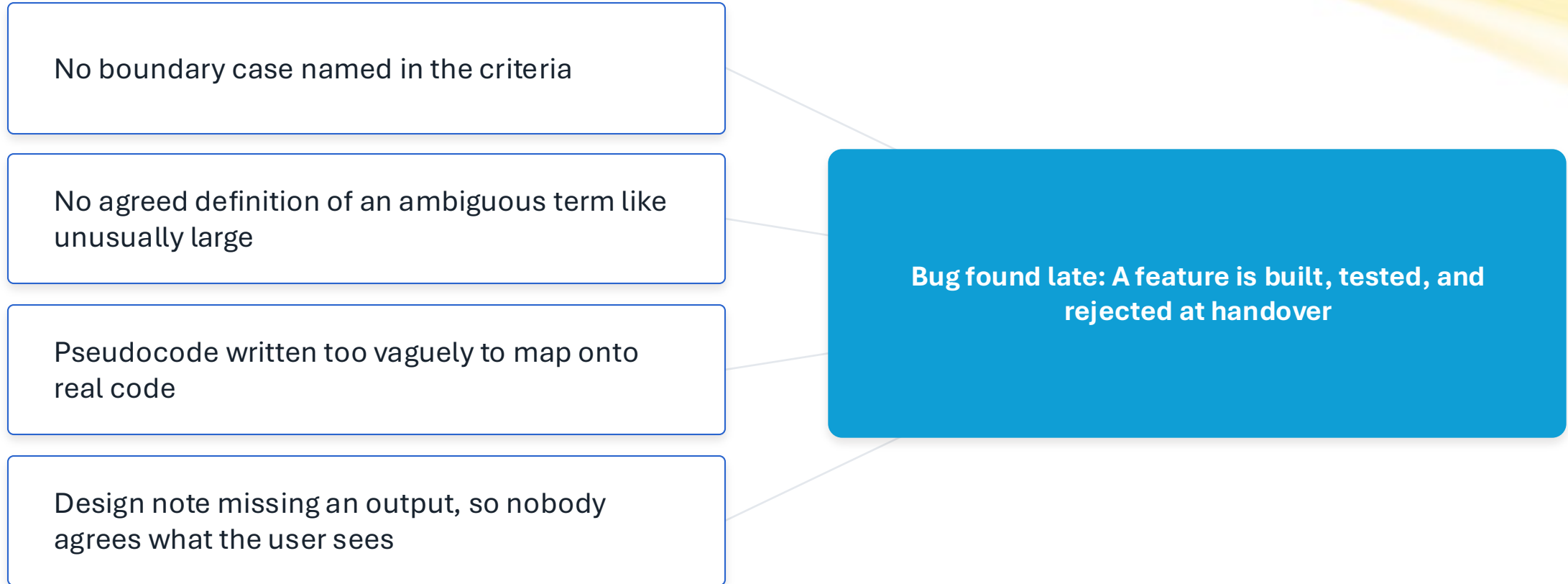
Too vague to act on

- This looks good
- Maybe check the criteria again

Specific enough to use

- Your boundary criterion for exactly 3 attempts is really clear
- Your second pseudocode line does not seem to check the same thing as your third criterion

What Slips Through Without a Design Stage



What You Can Now Produce

A quick stocktake of the planning artefacts you can now write with support.

- 1 A user story: naming a real user, their goal, and their reason.
- 2 Testable acceptance criteria: including a boundary case named explicitly.
- 3 Pseudocode: branching logic that maps onto if, else if and else you already know.
- 4 A design note: inputs, outputs, rules and notes, ready for another developer to follow.

Four Part Exit Ticket

Answer four short questions about today before you leave.

- 1 Name one part of a user story you sometimes forget to include.
- 2 Write one testable acceptance criterion for any feature of your choice.
- 3 Explain in one sentence why pseudocode is written before real code.
- 4 Name one thing from your planning pack you would check with a stakeholder.



TIMEBOX

10 minutes



OUTPUT

Four short written answers reflecting on today's planning skills.

Closing

You now have a proper planning process to bring into the project studio.



A user story names who a feature is for, what they want, and why.



Acceptance criteria define done clearly enough for someone else to test.



Pseudocode plans the logic before a single line of JavaScript is written.



A design note captures inputs, outputs and rules so anyone can pick up your plan.